Additional Experiment:

Containerising & Publishing to Docker Hub

Objective

To modify the existing automated pipeline so that every successful test run triggers the creation of a isolated **Docker image** containing your test framework, which is then securely pushed to your public/private **Docker Hub repository** upon a successful build.

Step 1: Create a Docker Hub Account & Repository

Before updating your code, you need a destination to host your container image.

1. Go to Docker Hub and sign in (or create an account).
2. Go to MyHub and Click **Create Repository**.
3. Name your repository selenium-testng-automation.
4. Set the visibility to **Public** and click **Create**.

Step 2: Generate a Docker Hub Access Token

For safety, you should never store your raw password in GitHub. Instead, use an Access Token.

1. In Docker Hub, click on your profile avatar in the top-right corner and select **Account Settings**.
2. Navigate to **Security** in the left sidebar.
3. Click **Personal Access Tokens** → **Generate New Token**.
4. Name it GitHub-Actions-CI and grant it **Read & Write** permissions.
5. **Copy the token immediately** (you will not see it again).

docker login -u dradityacr

dckr_pat_23j8bk-pMiUDEEvXyaBcTz7eXDA

Step 3: Save Credentials as GitHub Secrets

1. Open your **SeleniumLocalWebTest** repository on GitHub.
2. Navigate to **Settings** → **Secrets and variables** → **Actions**.
3. Click **New repository secret** and add the following two secrets:

- **Name:** DOCKERHUB_USERNAME
  Value: *Your actual Docker Hub username (eg : dradityacr)*

Name: DOCKERHUB_TOKEN
Value: *The personal access token you generated in Step 2 (eg:* dckr_pat_23j8k-pMiUDvXyaAcTz7eND)

Step 4: Create a Dockerfile in the Project Root

To bundle your Java project, you need a recipe blueprint file. Create a file named exactly Dockerfile (no extension) at the root level of your project structure:

```
SeleniumLocalWebTest/
│----- Dockerfile
├── pom.xml
├── testng.xml
├── webpage/
│   └── login.html
│
└── src/
    └── test/
        └── java/
            └── LocalLoginTest.java
```

Dockerfile Code

```
# ========================================

# STAGE 1: BUILD

# Maven + JDK 17

# ========================================

FROM maven:3.9.9-eclipse-temurin-17 AS build

WORKDIR /app

# Copy Maven configuration

COPY pom.xml .

# Copy source code

COPY src ./src
```

```
# Download dependencies

RUN mvn dependency:go-offline -B

# Compile Selenium TestNG test classes

RUN mvn test-compile

# ========================================

# STAGE 2: RUNTIME

# Maven + JDK 17 + Chromium

# ========================================

FROM maven:3.9.9-eclipse-temurin-17

WORKDIR /app

# Install Chromium

RUN apt-get update && apt-get install -y \

   chromium \

   && rm -rf /var/lib/apt/lists/*

# Copy project files

COPY --from=build /app /app

# Chromium configuration

ENV CHROME_BIN=/usr/bin/chromium

# Execute Selenium tests

CMD ["mvn", "test"]
```

Step 5: Extend the GitHub Actions Workflow File

Modify your .github/workflows/selenium-tests.yml file to include your Docker authentication, building steps, and target deployment push tasks:

```
name: Selenium Java Tests & Docker Publish

name: Selenium TestNG CI/CD Pipelin

# Trigger workflow on push and pull requests to main

on:

  push:

    branches: ["main"]
```

```
  pull_request:
    branches: ["main"]
jobs:
  # JOB 1: Execute Selenium TestNG Automation Tests
  selenium-test:
    runs-on: ubuntu-latest
    steps:
      # Step 1: Checkout source code
      - name: Checkout source code
        uses: actions/checkout@v4
      # Step 2: Set up JDK 17
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
          cache: maven
      # Step 3: Set up Chrome
      - name: Set up Chrome
        uses: browser-actions/setup-chrome@v1
      # Step 4: Verify Java and Chrome
      - name: Verify Java and Chrome
        run: |
          java -version
          google-chrome --version
      # Step 5: Run Selenium tests
      - name: Run Selenium tests
        run: mvn clean test
  # JOB 2: Build and Publish Docker Image
  # Runs only after successful Selenium tests
  build-and-publish-docker:
```

```
  needs: selenium-test
  runs-on: ubuntu-latest
  # Publish only on push events, not pull requests
  if: github.event_name == 'push'
  steps:
   # Step 1: Checkout source code
   - name: Checkout source code
     uses: actions/checkout@v4
   # Step 2: Login to Docker Hub
   - name: Log in to Docker Hub
     uses: docker/login-action@v3
     with:
       username: ${{ secrets.DOCKERHUB_USERNAME }}
       password: ${{ secrets.DOCKERHUB_TOKEN }}
   # Step 3: Set up Docker Buildx
   - name: Set up Docker Buildx
     uses: docker/setup-buildx-action@v3
   # Step 4: Build and Push Docker Image
   - name: Build and Push Docker Image
     uses: docker/build-push-action@v6
     with:
       context: .
       file: ./Dockerfile
       push: true
       tags: |
         ${{ secrets.DOCKERHUB_USERNAME }}/selenium-testng-automation:latest
         ${{ secrets.DOCKERHUB_USERNAME }}/selenium-testng-automation:${{ github.sha }}
```

Step 6: Commit and Push Changes

Save your files locally and use git commands to deploy the updates:

Git add . (or) git add Dockerfile .github/workflows/selenium-tests.yml

git commit -m "Extend CI workflow to package and push Docker image"

git push origin main

Step 7: Verification Checklist

1. **GitHub Actions:** Open the **Actions** tab in your GitHub repository. You will see a newly extended two-stage matrix workflow pipeline: selenium-test build-and-publish-docker.
2. **Docker Hub Registry:** Open your Docker Hub dashboard profile page. Under the selenium-testng-automation repository tags section, you will see a latest tag timestamped right now, along with a secondary tag representing your specific unique Git commit hash.